coursera

Hands-On-Learning: Scale to Million Users: Optimize GlobalAI's Production System

Learning Objectives

By the end of this activity, learners will be able to:

- LO1: Implement comprehensive monitoring for LLM systems using Prometheus and custom metrics.
- LO2: Create Grafana dashboards tracking performance, availability, usage, and cost metrics.
- LO3: Configure Kubernetes HPA auto-scaling based on queue depth and latency triggers.



- LO4: Deploy cost optimization strategies, including caching and intelligent model routing.
- LO5: Analyze performance testing results to validate scaling configurations.



Description

In this hands-on practice, you will implement production-grade monitoring, auto-scaling, and cost optimization for GlobalAI's LLM platform, achieving 99.95% uptime and 40-60% cost reduction while handling 10x traffic spikes.

Structure of the Activity

coursera

Title: Hands-on Practice: Implementing Monitoring, Auto-Scaling, and

Tools to Be Used:

- Prometheus for metrics collection
- Grafana for visualization and dashboards
- Kubernetes with HPA (Minikube or cloud provider)
- Apache JMeter for load testing
- Redis for response caching

coursera

Setup Requirements:

- Minikube or Kubernetes cluster access.
- Prometheus and Grafana (can use Helm charts for quick setup).
- Apache JMeter installed for load testing.
- Redis (can use Docker container).
- At least 16GB RAM for running complete monitoring stack.

Scenario

You are a Site Reliability Engineer at GlobalAI, an enterprise LLM platform with 500 corporate clients processing 50 million daily requests. Response times exceed 30 seconds (SLA: 2 seconds), monthly costs are $3 million with 30% utilization, and a Black Friday outage caused $10 million in lost revenue. You must achieve 99.95% uptime and 50% cost reduction.

Getting Started:

- Download and extract LLM-Deployment-Starter-Files.zip from the course resources.
- Navigate to the Module3_Monitoring folder and review M3_prometheus.yml and M3_grafana_dashboard.json.
- Analyze M3_traffic_patterns.csv to understand load patterns and cost spikes.
- Build monitoring, scaling, and optimization incrementally using the provided templates.

coursera

Activity 1: Implement Comprehensive Monitoring

Aligned LOs: LO1, LO2

Steps to Complete:

Step 1: Configure Prometheus Metrics

- Create prometheus.yml with 15-second scrape interval.
- Define custom metrics: request_latency_seconds, tokens_processed_total.
- Add gpu_utilization and cost_per_request metrics.
- Configure service discovery for LLM pods.

coursera

Step 2: Create Four-Dimensional Grafana Dashboard

- Build Performance panel: throughput (requests/sec), P50/P95/P99 latency.
- Build Availability panel: uptime percentage, error rate, health status.
- Build Usage panel: active users, requests by endpoint, token consumption.
- Build Cost panel: hourly spend, cost per request, projected monthly.

coursera

Step 3: Configure Alerting Rules

- Create alert for P95 latency > 1.5 seconds.
- Create an alert for an error rate > 1%.
- Create an alert for cost anomaly (>20% above baseline).
- Configure notification channels (Slack, PagerDuty).

coursera

Tips:

- Use histogram metrics for latency to enable percentile calculations.
- Include request_queue_depth metric—critical for scaling decisions.
- Set alert thresholds based on SLA requirements.

coursera

Activity 2: Configure Auto-Scaling with Load Testing

Aligned LOs: LO3, LO5

Steps to Complete:

Step 1: Create Kubernetes HPA Configuration

- Define HPA with custom metrics (not just CPU).
- Set triggers: request_queue_depth < 30, P95 latency < 1.5s.
- Configure min replicas: 2, max replicas: 10.

Step 2: Configure Asymmetric Scaling Policies

- Scale-up: Add two pods after 2 minutes of threshold breach.
- Scale-down: Remove one pod after 15 minutes of low utilization.
- Set stabilization window: 300 seconds to prevent flapping.

Step 3: Execute Load Testing with JMeter

- Create a JMeter test plan simulating a 10x traffic spike.
- Ramp from 100 to 1000 concurrent users over 5 minutes.
- Monitor scaling events in Kubernetes and Grafana.

Step 4: Analyze Results

- Verify latency stays under SLA during traffic spike.
- Confirm scaling events occurred at expected thresholds.
- Document scaling timeline and recovery time.

Tips:

- GPU instances take 4-5 minutes to boot—use queue depth for proactive scaling.
- Aggressive scale-up, conservative scale-down prevents oscillation.
- Test during low-traffic periods to avoid production impact.

coursera

Activity 3: Implement Cost Optimization

Aligned LO: LO4

Steps to Complete:

Step 1: Deploy Redis Caching

- Deploy a Redis cluster for response caching.
- Implement semantic similarity matching for cache hits.
- Configure TTL: 24 hours for general queries, 7 days for classifications.

Step 2: Implement Intelligent Model Routing

- Build a complexity classifier for incoming queries.
- Route simple queries (score < 30) to Llama 3.1-8B.
- Route complex queries (score >= 30) to Llama 3.1-70B.

Step 3: Configure Dynamic Scaling Schedule

- Set 3 instances during business hours (8 AM - 6 PM).
- Scale down to 1 instance during off-hours.
- Maintain auto-scaling capability for unexpected spikes.

Step 4: Measure Cost Reduction

- Calculate cache hit rate and savings.
- Measure cost per request before and after optimization.
- Project monthly savings based on traffic patterns.

Tips:

- Monitor cache hit rate—target 25-35% for good optimization.
- Validate routing accuracy to ensure complex queries get appropriate models.
- Balance cost savings against latency impact.

Compile Your Findings

Requirement 1: Prometheus Monitoring Configuration

Demonstrate comprehensive metrics collection for LLM operations.

Include:

- Prometheus configuration file (prometheus.yml).
- Custom metric definitions for LLM workloads.
- Service discovery configuration.
- Alerting rules with thresholds.

Requirement 2: Grafana Dashboard

Demonstrate four-dimensional observability for LLM systems.

Include:

- Grafana dashboard JSON export.
- Screenshots showing all four dimensions (performance, availability, usage, cost).
- Alert notification configuration.
- Explanation of metric choices and thresholds.

Requirement 3: Auto-Scaling Configuration

Demonstrate intelligent scaling that handles traffic spikes.

Include:

- Kubernetes HPA configuration (hpa.yaml).
- Custom metrics adapter configuration.
- Scaling event timeline during load test.
- Latency graphs demonstrating SLA compliance under load.

coursera

Requirement 4: Cost Optimization Implementation

Demonstrate significant cost reduction while maintaining performance.

Include:

- Redis caching configuration and hit rate metrics.
- Model routing logic and distribution analysis.
- Dynamic scaling schedule configuration.
- Cost comparison (before vs. after optimization).

Requirement 5: Performance Testing Results

Demonstrate validated scaling under realistic load conditions.

Include:

- JMeter test plan configuration.
- Load test results showing 10x traffic handling.
- Response time distribution under load.
- Business analysis (3-5 sentences): What cost reduction was achieved? Is 99.95% uptime achievable? What would you monitor going forward?

Summary

This activity provides hands-on experience with production LLM operations through practical implementation of monitoring, auto-scaling, and cost optimization. Participants have developed competency in Prometheus/Grafana observability, Kubernetes HPA configuration, and intelligent caching strategies, which are directly applicable to enterprise SaaS environments.

coursera

Key takeaways:

- Comprehensive monitoring across four dimensions (performance, availability, usage, cost) enables proactive operations and prevents costly outages.
- Auto-scaling based on queue depth and latency—rather than CPU utilization—is essential for GPU-based LLM infrastructure with 4-5 minute boot times.
- Cost optimization through caching and intelligent model routing can reduce LLM operational costs by 40-60% without sacrificing performance.

Resources

Primary Materials (from LLM-Deployment Repository files):

- M3_prometheus.yml - Full Prometheus configuration with scrape targets and custom metrics
- M3_alert_rules.yml - Alerting rules for latency, errors, availability, and cost
- M3_grafana_dashboard.json - Complete dashboard with SLA gauges and time series
- M3_hpa.yaml - Kubernetes HPA, VPA, and PDB configurations
- M3_cost_optimizer.py - Semantic caching and intelligent model routing module

- M3_traffic_patterns.csv - Sample traffic data with spike simulation
- M3_load_test.jmx - JMeter test plan for load testing
- M3_test_prompts.csv - Test prompts for load testing (simple, medium, complex)

Reference Documentation:

- Prometheus Documentation: https://prometheus.io/docs/
- Grafana Documentation: https://grafana.com/docs/
- Kubernetes HPA Documentation: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

Solution/Exemplar:

Hands_On_Practice_Module3_Solution

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.